

L14 — Records: Structure and Memory Representation; Parallel Arrays

Unit: 1 | Chapter: Pointers, Records & Advanced Arrays | BT Level: BT3 | CO: CO2

Learning Objectives

- Define a record (struct) and describe its memory layout
 - Calculate memory offsets and struct sizes including padding
 - Implement parallel arrays as an alternative representation
 - Compare structs vs parallel arrays in terms of cache performance
-

1. What is a Record?

A **record** (called `struct` in C/C++) is a composite data structure that groups together variables of **different types** under a single name. Each variable in a record is called a **field**.

```
struct Student {  
    int    roll_no;        // 4 bytes  
    char   name[50];       // 50 bytes  
    float  cgpa;           // 4 bytes  
    int    age;            // 4 bytes  
};
```

Records model real-world entities that have multiple attributes of different types.

2. Memory Representation of Records

2.1 Sequential Layout

Fields are stored **contiguously in memory** in declaration order (approximately).

```
struct Point { int x; int y; float z; };
```

Memory: [x: 4B][y: 4B][z: 4B] = 12 bytes total

2.2 Memory Alignment and Padding

Modern processors access memory most efficiently when data is aligned to its natural boundary:

- `int` (4B) → must be at address divisible by 4
- `double` (8B) → must be at address divisible by 8

The compiler inserts **padding bytes** to satisfy alignment requirements.

```

struct Example1 {
    char  a;    // 1 byte  → at offset 0
                // 3 bytes padding
    int   b;    // 4 bytes → at offset 4
    char  c;    // 1 byte  → at offset 8
                // 3 bytes padding
};
// sizeof(Example1) = 12 bytes (not 6!)

struct Example2 {
    int   b;    // 4 bytes → at offset 0
    char  a;    // 1 byte  → at offset 4
    char  c;    // 1 byte  → at offset 5
                // 2 bytes padding
};
// sizeof(Example2) = 8 bytes

```

Rule: Reordering fields from largest to smallest type minimizes padding.

2.3 Alignment Rules Summary

Type	Size	Alignment
char	1	1
short	2	2
int	4	4
float	4	4
double	8	8
pointer	8 (64-bit)	8

3. Array of Records (AoS)

An **Array of Structs** stores all fields of each record together:

```

struct Student {
    int id;
    char name[20];
    float score;
};

struct Student students[100]; // Array of 100 Student records

// Access:
students[5].id = 101;
students[5].score = 9.5;

```

Memory layout:

```
[id0|name0|score0][id1|name1|score1]...[id99|name99|score99]
```

4. Parallel Arrays (SoA — Struct of Arrays)

Parallel arrays store each field of the records in separate arrays of the same length, indexed in parallel:

```
int    ids[100];
char   names[100][20];
float  scores[100];

// Access same logical record at index i:
ids[5] = 101;
scores[5] = 9.5;
```

Or as a struct of arrays:

```
struct StudentsDB {
    int    ids[100];
    char   names[100][20];
    float  scores[100];
} db;
```

Memory layout:

```
[id0, id1, ..., id99][name0, name1, ..., name99][score0, ..., score99]
```

5. AoS vs SoA: Performance Comparison

Criterion	Array of Structs (AoS)	Parallel Arrays (SoA)
Field access (single record)	Excellent (all fields nearby)	Poor (scattered)
Processing single field across all records	Poor	Excellent (contiguous)
Cache efficiency	Record-level	Field-level
SIMD vectorization	Difficult	Easy
Code clarity	Better	More verbose

Example: Computing average score of all students:

```
# AoS (array of dicts in Python)
students_aos = [{"id": i, "name": f"S{i}", "score": float(i)} for i in range(1000)]
avg = sum(s["score"] for s in students_aos) / len(students_aos)

# SoA (parallel arrays)
ids = list(range(1000))
names = [f"S{i}" for i in range(1000)]
scores = [float(i) for i in range(1000)]
avg = sum(scores) / len(scores)    # Only 'scores' array is accessed - c
```

When only scores is needed, SoA avoids loading id and name into cache.

6. Practical Examples in Python

Using Dictionaries as Records

```
student = {
    "roll_no": 1,
    "name": "Alice",
    "cgpa": 9.2,
    "age": 20
}
print(student["name"])  # Alice
```

Using Named Tuples (Immutable Records)

```
from collections import namedtuple

Student = namedtuple("Student", ["roll_no", "name", "cgpa", "age"])
alice = Student(1, "Alice", 9.2, 20)
print(alice.name)  # Alice
print(alice[2])    # 9.2
```

Using dataclasses (Modern Python)

```
from dataclasses import dataclass

@dataclass
class Student:
    roll_no: int
    name: str
    cgpa: float
    age: int

alice = Student(1, "Alice", 9.2, 20)
print(alice)  # Student(roll_no=1, name='Alice', cgpa=9.2, age=20)
```

7. Sorting Records

```
students = [
    {"name": "Charlie", "cgpa": 8.5},
    {"name": "Alice",    "cgpa": 9.2},
    {"name": "Bob",      "cgpa": 8.9},
]

# Sort by CGPA descending
students.sort(key=lambda s: s["cgpa"], reverse=True)
# [Alice(9.2), Bob(8.9), Charlie(8.5)]
```

Summary

- A **record** (struct) groups fields of different types into one logical unit.
- Memory layout involves **padding** to satisfy alignment requirements — reorder fields

(large to small) to minimize wasted bytes.

- **Array of Structs (AoS):** Best when accessing all fields of individual records.
 - **Parallel Arrays (SoA):** Best when processing a single field across all records (better SIMD/vectorization support).
 - Python models records with dictionaries, namedtuples, or dataclasses.
-

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)